



SECURITY ASSESSMENT Volt Inu Heavy Pool

October 2, 2026

Audit Status: Pass

BLADE POOL

RISK ANALYSIS | Volt Inu Heavy.

■ Classifications of Manual Risk Results

Note: Risk classifications follow CVSS v4.0 standards (<https://www.first.org/cvss/v4.0/specification-document>). Contract security metrics are verified through GoPlus API token scanner technology, which provides real-time validation of contract parameters including

Classification	CVSS Range	Description
 Critical	9.0 - 10.0	Critical vulnerabilities that pose immediate and severe risks requiring urgent attention.
 High	7.0 - 8.9	High-priority issues that could lead to significant security breaches or financial loss.
 Medium	4.0 - 6.9	Medium-severity findings that should be addressed to improve contract security.
 Low	0.1 - 3.9	Low-risk items or best practice suggestions with minimal security impact.
 Informational	0.0	Informational findings about detected functions or contract features.

■ Manual Code Review Risk Results

Contract Security	Description
 Buy Tax	N/A
 Sale Tax	N/A
 Cannot Buy	N/A
 Cannot Sale	N/A
 Max Tax	30% early-unstake penalty
 Modify Tax	No - penalty is a constant
 Fee Check	30% early-unstake penalty (constant)
 Is Honeypot?	Not Detected
 Trading Cooldown	Lock period (owner-set, 1 to 60 days)
 Enable Trade?	N/A

Contract Security	Description
● Pause Transfer?	Not Detected
● Max Tx?	Per-wallet max stake
● Is Anti Whale?	Per-wallet max stake
● Is Anti Bot?	Not Detected
● Is Blacklist?	Not Detected
● Blacklist Check	Not Detected
● Is Whitelist?	Not Detected
● Can Mint?	No mint capability
● Is Proxy?	Not Detected
● Can Take Ownership?	No - ownership is fixed (no transfer/renounce)
● Hidden Owner?	Not Detected
● Owner	0x8Ed4A6306088994CDb1CAcEf35B2Ac4155F29f47
● Self Destruct?	Not Detected
● External Call?	Present (VOLT token)
● Other?	Round-based VOLT staking; pro-rata reward pot; 30% early-unstake penalty
● Holders	N/A (staking contract)
● Audit Confidence	Good - Pass (preliminary; principal protected from owner)
● Authority Check	Owner-controlled
● Freeze Check	Not Detected

Overview of identified vulnerabilities and risks. See the full report for detailed methodology and recommendations.

CFG Ninja Verified on October 2, 2026



Volt Inu Heavy

Executive Summary

TYPES

DeFi

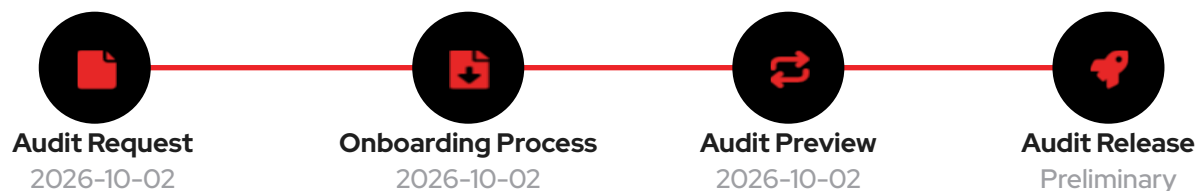
ECOSYSTEM

Robinhood Chain

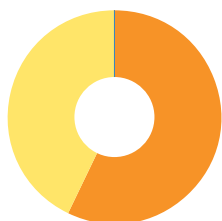
LANGUAGE

Solidity

Timeline



Vulnerability Summary



7
Total Findings

0
Resolved

7
Pending

7
Unresolved

19 of 26 CFG01-CFG26 checks passed with no security finding; 7 open findings. Slither 0.11.5 static analysis.

Severity	Resolution Status	Description
0 Critical		Immediate danger. Can lead to loss of user funds, unauthorized access, or full system compromise.
0 High		Significant risk that should be addressed urgently to prevent exploitation or financial loss.
4 Medium	0 Resolved, 4 Pending	Moderate risk that can lead to issues if combined with other weaknesses; should be addressed.
3 Low	0 Resolved, 3 Pending	Minor concern or best-practice deviation with limited direct security impact.
0 Informational		Not a vulnerability - code-quality, documentation, or optimization suggestions.

Industry Standards Compliance

This audit follows internationally recognized security standards to provide comprehensive assurance.

EEA EthTrust Security Levels

EthTrust/EVM review conducted for a Robinhood Chain contract.

Smart Contract Security Verification Standard (SCSVS)

SCSVS review conducted for an EVM staking contract.

CVSS v3.1 Severity Scoring

Per-finding CVSS recorded on each CFG item.

PROJECT OVERVIEW | Volt Inu Heavy.

Token Summary

Parameter	Result
Address	0x69275a8ec97289fef275999e06d61c9d1a923567
Name	Volt Inu Heavy
Token Tracker	Volt Inu Heavy (VOLT)
Decimals	18
Supply	1,000,000,000 (VOLT)
Platform	Robinhood Chain
Compiler	v0.8.34+commit.80d5c536
Contract Name	VoltEcoHeavy
Optimization	Yes with 200 runs
LicenseType	MIT (SPDX)
Language	Solidity
Codebase	https://robinhoodchain.blockscout.com/address/0x69275a8ec97289fef275999e06d61c9d1a923567?tab=contract

■ Main Contract Assessed

Name	Contract	Live
Volt Inu Heavy	0x69275a8ec97289fef275999e06d61c9d1a923567	Yes
VOLT token (staked)	0x5d102d1E69E77591D486aa4f663B3645151BBdf6	Yes
VoltEcoLite (sister pool)	0x94f70f580fc47cb25fe0a0424a5605bc3f07ab0f	Yes

■ TestNet Contract Was Not Assessed

■ Solidity Code Provided

SoIID	File Sha-1	FileName
VoltEcoHeavy	a0de0961b1842c067774968543711e8b13073107	heavy.sol

TECHNICAL FINDINGS

Smart contract security audits classify risks into several categories: Critical, High, Medium, Low, and Informational. These classifications help assess the severity and potential impact of vulnerabilities found in smart contracts.

Classification of Risk

Severity	CVSS Range	Description
 Critical	9.0 - 10.0	Immediate danger. Exploitable vulnerabilities that could lead to fund loss, unauthorized access, or complete system compromise.
 High	7.0 - 8.9	Significant security risk. Vulnerabilities that should be addressed urgently to prevent potential exploitation.
 Medium	4.0 - 6.9	Moderate security risk. Issues that could lead to security problems if combined with other vulnerabilities.
 Low	0.1 - 3.9	Minor security concern. Best practice violations or low-impact issues.
 Informational	0.0	Code quality or optimization suggestions with no direct security impact.

Slither Static Analysis Results

Slither is a Solidity static analysis framework that runs a suite of vulnerability detectors to identify potential security issues, code quality problems, and best practice violations. It provides comprehensive security analysis with confidence ratings for each finding. For full detector documentation, visit: <https://github.com/crytic/slither/wiki/Detector-Documentation>

Impact Level	Count	Description
High	0	Critical security vulnerabilities requiring immediate attention (e.g., reentrancy, unchecked transfers)
Medium	9	Significant issues that should be addressed (e.g., dangerous comparisons, arithmetic issues)
Low	17	Minor issues and best practice violations (e.g., missing events, naming conventions)
Informational	3	Code quality and optimization suggestions (e.g., unused variables, gas optimizations)
TOTAL	29	Total findings across all severity levels

Analysis Summary

Slither (101 detectors, solc 0.8.34) reported 29 results: 0 High; 9 Medium (6 strict-equality checks that are intentional zero guards, 1 uninitialized local that defaults to zero, 2 reentrancy-no-eth on transferFrom before state updates, covered in CFG19 and not exploitable with VOLT); 17 Low (8 timestamp, 7 reentrancy-benign/ events, 2 missing events on admin withdrawals, see CFG05); 3 Optimization (owner/penaltyWallet/ devWallet could be immutable).

Remediation Approach

All High and Medium impact findings from Slither have been manually reviewed and mapped to CFG test cases where applicable. Each finding has been analyzed in the context of the contract's business logic to determine actual exploitability. Detailed analysis of specific findings is available in the Technical Findings section of this audit report.

Deprecation Notice

The Smart Contract Weakness Classification Registry (SWC Registry) was deprecated in September 2023. CFGNinja audits now use Slither for static analysis, which provides more comprehensive and up-to-date vulnerability detection. SWC was an implementation of the weakness classification scheme proposed in EIP-1470, loosely aligned to the Common Weakness Enumeration (CWE). For historical reference: <https://swcregistry.io/>

Detailed Security Findings

Each finding lists its severity, CVSS, location and description, the affected source excerpt (verbatim from the verified Blockscout source), our recommendation, mitigation, references, and a suggested fix where the remediation is code-expressible.

VOLT-04 | New Round Forfeits Unclaimed Rewards.

Category	Severity	CVSS	Location	Status
Centralization / Denial-of-Service	 Medium	4.3	heavy.sol: startPool()	 Detected

Description

claim() is keyed to the current roundId. startPool() can be called by the owner the moment a round ends; it increments roundId, resets frozenTotal and rolls the whole pot into the next round. Any eligible staker who has not claimed yet loses that round's reward with no grace period. With a 1-day lock currently set, the claim window can be seconds.

Recommendation

Track pot and claims per round so earlier rounds stay claimable, or enforce a minimum claim window before startPool() can run again.

Mitigation

Open - pending client response.

References:

<https://swcregistry.io/docs/SWC-116>

<https://swcregistry.io/docs/SWC-105>

<https://owasp.org/www-project-smart-contract-top-10/>

VOLT-05 | Missing Events on Fund Withdrawals.

Category	Severity	CVSS	Location	Status
Logging	 Informational	0.0	heavy.sol: recoverTax() / rescue()	 Detected

Description

recoverTax() and rescue() move tokens without emitting events, which hides reward withdrawals from off-chain monitoring.

Recommendation

Emit events for every owner withdrawal.

Mitigation

Disclosure only.

References:

N/A

VOLT-08 | Unused devWallet and Fixed Ownership.

Category	Severity	CVSS	Location	Status
Code Quality	 Informational	0.0	heavy.sol: devWallet / owner	 Detected

Description

devWallet is set but never used. owner has no transfer or renounce function, so a lost or compromised key cannot be rotated.

Recommendation

Remove unused state and add two-step ownership transfer.

Mitigation

Disclosure only.

References:

N/A

VOLT-16 | Owner Can Withdraw the Reward Pot.

Category	Severity	CVSS	Location	Status
Centralization	 Medium	5.0	heavy.sol: recoverTax() / rescue()	 Detected

Description

recoverTax() and rescue(volt) let the owner transfer any VOLT above totalStaked, which is exactly the reward pot and pending rewards, including a pot that eligible stakers are in the middle of claiming. Neither function emits an event. Staked principal is protected (amount <= _freeBalance()), but rewards shown in preview() are not guaranteed.

Recommendation

Limit recoverTax/rescue to non-VOLT tokens or to balance not yet committed to pot/pending, add a timelock and emit events.

Mitigation

Open - pending client response.

References:

<https://swcregistry.io/docs/SWC-105>

<https://owasp.org/www-project-smart-contract-top-10/>

VOLT-18 | 30% Early-Unstake Penalty Paid to Deployer.

Category	Severity	CVSS	Location	Status
Transparency	 Low	2.0	heavy.sol: unstake() / penaltyWallet	 Detected

Description

Unstaking before unlockAt while the pool is live costs 30% (PENALTY_BPS = 3000), sent to penaltyWallet, which is hardcoded to the deployer address and cannot be changed. The penalty does not go to the remaining stakers. The penalty also does not apply after stopPool(), so the owner decides when exits are free.

Recommendation

Disclose the penalty and its recipient clearly, or route penalties to the reward pot.

Mitigation

Open - pending client response.

References:

<https://consensys.github.io/smart-contract-best-practices/>

VOLT-19 | External Call Before State Update Without Guard.

Category	Severity	CVSS	Location	Status
Reentrancy	 Low	2.0	heavy.sol: stake() / unstake() / claim()	 Detected

Description

stake() calls volt.transferFrom() before updating staked/totalStaked, and there is no reentrancy guard. VOLT is a verified token with no transfer hooks, so this is not exploitable today, but the pattern is unsafe if the token were ever changed or reused with a hook-bearing token.

Recommendation

Follow checks-effects-interactions and add a nonReentrant guard to stake, unstake, claim and admin transfer functions.

Mitigation

Not exploitable with the current VOLT token.

References:

<https://swcregistry.io/docs/SWC-107>

VOLT-20 | Owner Can Change Eligibility Mid-Round.

Category	Severity	CVSS	Location	Status
Centralization / Ownership	 Medium	5.0	heavy.sol: setLimits() / setLockTime() / stopPool()	 Detected

Description

A single EOA owner (0x8Ed4...9f47, not transferable or renounceable) can raise minStake after a round ends to disqualify stakers before they claim (eligible() checks the current minStake), change lockTime and therefore the hold time retroactively, or call stopPool(), after which eligible() skips the hold check entirely. At block 78,011,482 the live settings differ from the published site: Heavy minStake and maxStake are both 110,000 VOLT (code defaults 2M and 20M; site says 2M+) and lockTime is 1 day. Stakers must trust the owner not to change rules mid-round.

Recommendation

Apply limit and lock changes only from the next round, snapshot eligibility rules at startPool(), move ownership to a multisig and publish parameter changes.

Mitigation

Open - pending client response.

References:

<https://swcregistry.io/docs/SWC-116>

<https://swcregistry.io/docs/SWC-105>

<https://owasp.org/www-project-smart-contract-top-10/>

VOLT-22 | Last-Minute Top-Up Captures Round Rewards.

Category	Severity	CVSS	Location	Status
Manipulation Prevention	 Medium	5.3	heavy.sol: stake() / claim() / eligible()	 Detected

Description

Rewards are paid pro-rata on the stake a wallet holds when it claims, but the hold-time requirement only applies to the moment a wallet first reaches minStake (qualifiedAt is never reset by top-ups). A wallet can qualify early with the minimum, then top up to maxStake (20,000,000 VOLT) just before roundEnd and claim a share sized on the top-up. A top-up does not extend unlockAt if the existing lock is still running, so the extra stake can usually be withdrawn shortly after the round without the 30% penalty. Honest long-term stakers are diluted.

Recommendation

Use time-weighted stake, or snapshot each wallet's balance at the point it becomes eligible and pay on the minimum balance held through the round. Reset the lock on every top-up.

Mitigation

Open - pending client response.

References:

<https://swcregistry.io/docs/SWC-116>

<https://swcregistry.io/docs/SWC-105>

<https://owasp.org/www-project-smart-contract-top-10/>

VOLT-23 | Anonymous Team - No KYC Verification.

Category	Severity	CVSS	Location	Status
Team Transparency	 Informational	0.0	heavy.sol: Project team	 Detected

Description

No team identities are disclosed and no KYC has been completed with a recognized provider. Flagged for transparency; the score cap of 90 applies. It does not, by itself, indicate a code vulnerability.

Recommendation

Complete KYC with a recognized provider.

Mitigation

No KYC found.

References:

N/A

VOLT-24 | Qualification Not Refreshed After Limit Changes.

Category	Severity	CVSS	Location	Status
Input Validation	 Low	2.5	heavy.sol: setLimits() / stake() / unstake()	 Detected

Description

qualifiedAt is only set inside stake() and cleared inside unstake(). If the owner lowers minStake, existing stakers who now meet it keep qualifiedAt = 0 and stay ineligible while the pool is live until they stake again. Raising minStake does not clear qualifiedAt, so a later top-up reuses the old qualification time.

Recommendation

Derive qualification from the current balance and a stored timestamp on read, or refresh qualification when limits change.

Mitigation

Open - pending client response.

References:

<https://swcregistry.io/docs/SWC-105>

<https://owasp.org/www-project-smart-contract-top-10/>

FINDINGS

In this document, we present the findings and results of the smart contract security audit. The identified vulnerabilities, weaknesses, and potential risks are outlined, along with recommendations for mitigating these issues. It is crucial for the team to address these findings promptly to enhance the security and trustworthiness of the smart contract code.

Severity	Found	Pending	Resolved
 Critical	0	0	0
 High	0	0	0
 Medium	4	4	0
 Low	3	3	0
 Informational	0	0	0
Total	7	7	0

In a smart contract, a technical finding summary refers to a compilation of identified issues or vulnerabilities discovered during a security audit. These findings can range from coding errors and logical flaws to potential security risks. It is crucial for the project owner to thoroughly review each identified item and take necessary actions to resolve them. By carefully examining the technical finding summary, the project owner can gain insights into the weaknesses or potential threats present in the smart contract. They should prioritize addressing these issues promptly to mitigate any risks associated with the contract's security. Neglecting to address any identified item in the security audit can expose the smart contract to significant risks. Unresolved vulnerabilities can be exploited by malicious actors, potentially leading to financial losses, data breaches, or other detrimental consequences. To ensure the integrity and security of the smart contract, the project owner should engage in a comprehensive review process. This involves understanding the nature and severity of each identified item, consulting with experts if needed, and implementing appropriate fixes or enhancements. Regularly updating and maintaining the smart contract's codebase is also essential to address any emerging security concerns. By diligently reviewing and resolving all identified items in the technical finding summary, the project owner can significantly reduce the risks associated with the smart contract and enhance its overall security posture.

SOCIAL MEDIA CHECKS | Volt Inu Heavy.

Social Media	URL	Result
Website	https://voltinu.meme/eco	Pass
Telegram	https://t.me/voltinuRH	Pass
Twitter	https://x.com/voltinu_rh	Pass
Facebook		N/A
Reddit	N/A	N/A
Instagram	N/A	N/A
CoinGecko		N/A
Github	N/A	N/A
CMC		N/A
Email		Contact
Other		N/A

From a security assessment standpoint, inspecting a project's social media presence is essential. It enables the evaluation of the project's reputation, credibility, and trustworthiness within the community. By analyzing the content shared, engagement levels, and the response to any security-related incidents, one can assess the project's commitment to security practices and its ability to handle potential threats.

Social Media Information Notes:

Auditor Notes: Project site voltinu.meme/eco confirmed, linking X ([voltinu_rh](https://x.com/voltinu_rh)) and Telegram ([voltinuRH](https://t.me/voltinuRH)). No CoinGecko/CMC listing checked. Team is anonymous; no KYC.

Project Owner Notes: Public site and community channels confirmed.

Assessment Results**Final Audit Score VOLT.**

Review	Score
Security Score	85
Auditor Score	85

Our security assessment or audit score system for the smart contract and project follows a comprehensive evaluation process to ensure the highest level of security. The system assigns a score based on various security parameters and benchmarks, with a passing score set at 80 out of a total attainable score of 100. The assessment process includes a thorough review of the smart contracts codebase, architecture, and design principles. It examines potential vulnerabilities, such as code bugs, logical flaws, and potential attack vectors. The evaluation also considers the adherence to best practices and industry standards for secure coding. Additionally, the system assesses the projects overall security measures, including infrastructure security, data protection, and access controls. It evaluates the implementation of encryption, authentication mechanisms, and secure communication protocols. To achieve a passing score, the smart contract and project must attain a minimum of 80 points out of the total attainable score of 100. This ensures that the system has undergone a rigorous security assessment and meets the required standards for secure operation.



Important Notes for VOLT

Volt Inu - VoltEco Heavy Pool - Smart Contract Security Audit Report (PRELIMINARY)

PROJECT OVERVIEW

VoltEco Heavy is one of two Volt Inu staking pools on Robinhood Chain. Holders stake VOLT (default 2,000,000 to 20,000,000 per wallet) for an owner-set lock; each round, stakers who held at least the minimum for the hold period share a pot of VOLT deposited into the pool, pro-rata to their stake. Unstaking early while the pool is live costs 30%.

PRELIMINARY report; findings open pending client response. The Lite pool shares this code (except limits) and has its own report.

CONTRACT DETAILS

Name: VoltEcoHeavy (heavy.sol)

Address: 0x69275a8ec97289fef275999e06d61c9d1a923567

Chain: Robinhood Chain (EVM, chainId 4663)

Compiler: Solidity v0.8.34+commit.80d5c536 | Optimizer: 200 runs | License: MIT (SPDX)

Verification: Sourcify exact match

Owner / Deployer / Penalty wallet: 0x8Ed4A6306088994CDb1CAcEf35B2Ac4155F29f47 (EOA)

Deployed: 2026-09-29

Staked token: VOLT 0x5d102d1E69E77591D486aa4f663B3645151BBdf6

Website: <https://voltinu.meme/eco>

SECURITY FINDINGS

New Round Forfeits Unclaimed Rewards (MEDIUM - CFG04)

claim() is keyed to the current roundId. startPool() can be called by the owner the moment a round ends; it increments roundId, resets frozenTotal and rolls the whole pot into the next round. Any eligible staker who has not claimed yet loses that round's reward with no grace period. With a 1-day lock currently set, the claim window can be seconds.

Recommendation: Track pot and claims per round so earlier rounds stay claimable, or enforce a minimum claim window before startPool() can run again.

Owner Can Withdraw the Reward Pot (MEDIUM - CFG16)

recoverTax() and rescue(volt) let the owner transfer any VOLT above totalStaked, which is exactly the reward pot and pending rewards, including a pot that eligible stakers are in the middle of claiming. Neither function emits an event. Staked principal is protected (amount <= _freeBalance()), but rewards shown in preview() are not guaranteed.

Recommendation: Limit recoverTax/rescue to non-VOLT tokens or to balance not yet committed to pot/pending, add a timelock and emit events.

Owner Can Change Eligibility Mid-Round (MEDIUM - CFG20)

A single EOA owner (0x8Ed4...9f47, not transferable or renounceable) can raise minStake after a round ends to disqualify stakers before they claim (eligible() checks the current minStake), change lockTime and therefore the hold time retroactively, or call stopPool(), after which eligible() skips the hold check entirely. At block 78,011,482 the live settings differ from the published site: Heavy minStake and maxStake are both 110,000 VOLT (code defaults 2M and 20M; site says 2M+) and lockTime is 1 day. Stakers must trust the owner not to change rules mid-round.

Recommendation: Apply limit and lock changes only from the next round, snapshot eligibility rules at startPool(), move ownership to a multisig and publish parameter changes.

Last-Minute Top-Up Captures Round Rewards (MEDIUM - CFG22)

Rewards are paid pro-rata on the stake a wallet holds when it claims, but the hold-time requirement only applies to the moment a wallet first reaches minStake (qualifiedAt is never reset by top-ups). A wallet can qualify early with the minimum, then top up to maxStake (20,000,000 VOLT) just before roundEnd and claim a share sized on the top-up. A top-up does not extend unlockAt if the existing lock is still running, so the extra stake can usually be withdrawn shortly after the round without the 30% penalty. Honest long-term stakers are diluted.

Recommendation: Use time-weighted stake, or snapshot each wallet's balance at the point it becomes

eligible and pay on the minimum balance held through the round. Reset the lock on every top-up.

30% Early-Unstake Penalty Paid to Deployer (LOW - CFG18)

Unstaking before unlockAt while the pool is live costs 30% ($\text{PENALTY_BPS} = 3000$), sent to penaltyWallet, which is hardcoded to the deployer address and cannot be changed. The penalty does not go to the remaining stakers. The penalty also does not apply after stopPool(), so the owner decides when exits are free.

Recommendation: Disclose the penalty and its recipient clearly, or route penalties to the reward pot.

External Call Before State Update Without Guard (LOW - CFG19)

stake() calls volt.transferFrom() before updating staked/totalStaked, and there is no reentrancy guard. VOLT is a verified token with no transfer hooks, so this is not exploitable today, but the pattern is unsafe if the token were ever changed or reused with a hook-bearing token.

Recommendation: Follow checks-effects-interactions and add a nonReentrant guard to stake, unstake, claim and admin transfer functions.

Qualification Not Refreshed After Limit Changes (LOW - CFG24)

qualifiedAt is only set inside stake() and cleared inside unstake(). If the owner lowers minStake, existing stakers who now meet it keep qualifiedAt = 0 and stay ineligible while the pool is live until they stake again. Raising minStake does not clear qualifiedAt, so a later top-up reuses the old qualification time.

Recommendation: Derive qualification from the current balance and a stored timestamp on read, or refresh qualification when limits change.

Missing Events on Fund Withdrawals (INFORMATIONAL - CFG05)

recoverTax() and rescue() move tokens without emitting events, which hides reward withdrawals from off-chain monitoring.

Recommendation: Emit events for every owner withdrawal.

Unused devWallet and Fixed Ownership (INFORMATIONAL - CFG08)

devWallet is set but never used. owner has no transfer or renounce function, so a lost or compromised key cannot be rotated.

Recommendation: Remove unused state and add two-step ownership transfer.

Anonymous Team - No KYC Verification (INFORMATIONAL - CFG23)

No team identities are disclosed and no KYC has been completed with a recognized provider. Flagged for transparency; the score cap of 90 applies. It does not, by itself, indicate a code vulnerability.

Recommendation: Complete KYC with a recognized provider.

POSITIVE FINDINGS

Principal Protection: owner withdrawals are limited to the balance above totalStaked; staked VOLT cannot be taken.

Bounded Parameters: min/max stake and lock time are capped by constants (lock 1 to 60 days).

Untaxed Transfers: VOLT charges its 2% tax only on DEX trades, so stake and unstake amounts are exact.

Solvent Payouts: claims are capped at the free balance, so rewards can never consume principal.

AUDIT METHODOLOGY

Manual line-by-line review of the verified Sourcify source against the CFG01-CFG26 framework, Slither static analysis (29 results triaged; no High) and on-chain state reads over the Robinhood Chain RPC.

Audit Date: October 2, 2026 | Chain: Robinhood Chain | Auditor: CFG.NINJA | Status: PRELIMINARY

DISCLAIMER

This audit does not guarantee the absence of all vulnerabilities. It covers the VoltEcoHeavy contract only. Users should conduct their own research before interacting with any smart contract.

I Appendix

Finding Categories

Centralization / Privilege

Centralization / Privilege findings refer to either feature logic or implementation of components that act against the nature of decentralization, such as explicit ownership or specialized access roles in combination with a mechanism to relocate funds.

Gas Optimization

Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction in the total gas cost of a transaction.

Logical Issue

Logical Issue findings detail a fault in the logic of the linked code, such as an incorrect notion of how `block.timestamp` works.

Control Flow

Control Flow findings concern the access control imposed on functions, such as owner-only functions being invocable by anyone under certain circumstances.

Volatile Code

Volatile Code findings refer to segments of code that behave unexpectedly in certain edge cases that may result in a vulnerability.

Coding Style

Coding Style findings usually do not affect the generated byte-code but rather comment on how to make the codebase more legible and, as a result, easily maintainable.

Inconsistency

Inconsistency findings refer to functions that should seemingly behave similarly yet contain different code, such as a constructor assignment imposing different require statements on the input variables than a setter function.

Coding Best Practices

ERC 20 Coding Standards are a set of rules that each developer should follow to ensure the code meets a set of criteria and is readable by all developers.

Disclaimer

Scope. This report documents the results of a security assessment and smart contract audit performed by Bladepool / CFG NINJA (the "Auditor") for the project specified in this report. The assessment covers only the deliverables and code expressly listed in the engagement; any changes, integrations, or subsequent deployments are outside the scope.

No Warranty. The Auditor performs the assessment using industry-standard practices but makes no representations or warranties, express or implied, including any warranty of merchantability, fitness for a particular purpose, or non-infringement. The Auditor does not guarantee that the audited code is free of bugs, vulnerabilities, or exploitable issues.

Limitation of Liability. To the maximum extent permitted by law, the Auditor and its affiliates, officers, employees, agents, and contractors shall not be liable for any indirect, incidental, special, consequential, or punitive damages, or for any loss of profits, revenue, data, or business opportunities, arising out of or related to the assessment, even if advised of the possibility of such damages. The Auditor's aggregate liability for direct damages is limited to the fees paid for the audit engagement.

Client Responsibility. The project owner/client retains sole responsibility for all decisions and actions taken in connection with the audited code, including fixing identified issues, deploying code to any environment, and applying security mitigations. The Auditor's recommendations are advisory; implementation and verification of fixes are the client's responsibility.

Third-Party Tools and Services. The Auditor may use third-party tools, services, or automated scanners as part of the assessment. Use of such tools is without warranty; the Auditor is not responsible for defects, failures, or inaccuracies arising from third-party services.

Confidentiality and Data Security. The Auditor will treat non-public client information as confidential. However, the Auditor cannot guarantee absolute security for data transmitted over the internet or third-party platforms. Client should take reasonable precautions to protect sensitive information.

Not Financial, Legal, or Investment Advice. The assessment is a technical security review only and is not financial, legal, tax, or investment advice. Clients should consult appropriate professionals for those matters.

Ownership and Governance Notes. If ownership is transferred, held by a multisig, or controlled by a governance contract, the client should document and disclose those arrangements; the Auditor's report reflects the ownership state observed during the engagement but may not reflect subsequent changes.

Governing Law and Counsel. This engagement and any disputes arising from it shall be governed by the laws agreed in the engagement terms. Clients are advised to seek legal counsel before relying on or publishing the report.

Acceptance. By proceeding with this engagement or using this report, the client acknowledges and accepts these terms.

